

1. SQL BASICS

★ 1) What is Database ?

A Database is an organized collection of data so that it can be easily stored, managed, retrieved and updated.



★ 2) What is DBMS ?

DBMS (Database Management System) is software that helps to create, store, manage and retrieve data from databases.

★ 3) What is RDBMS ?

RDBMS (Relational Database Management System) stores data in the form of tables (rows and columns) and the tables are related to each other.

Examples : MySQL, PostgreSQL, Oracle, SQL Server, SQLite, MariaDB, etc.

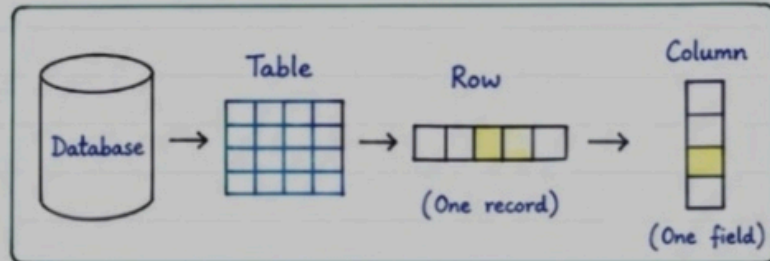
★ 4) What is SQL ?

SQL (Structured Query Language) is a standard language used to communicate with relational databases. It is used for creating, reading, updating, deleting data and for analysis/reporting.

5) SQL Command Types

1. DDL - Data Definition Language
2. DML - Data Manipulation Language
3. DCL - Data Control Language
4. TCL - Transaction Control Language
5. DQL - Data Query Language

6) Data Hierarchy in SQL



7) Example Table : Employee

ID	Name	Age	Dept	Salary
1	Abhi	22	IT	50000
2	Ravi	23	HR	45000
3	Sita	21	IT	52000

8) SQL vs MySQL

SQL	MySQL
- SQL is a language. - It is used to communicate with databases. - Example Commands: SELECT, INSERT, UPDATE, DELETE, etc.	- MySQL is a Database Management System. - It is an open-source RDBMS. - It uses SQL language to manage data.

9) Remember / Quick Tip

- SQL is the language.
- MySQL is the software (DBMS) that uses SQL. 😊

10) Interview Key Points / Shortcuts

(1) SQL Clause Order (Logical) FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT	(2) Common Data Types INT, BIGINT, VARCHAR(n), CHAR(n), DATE, DATETIME, DECIMAL(p,s), TEXT, BOOL	(3) Null vs Blank NULL = Unknown / Missing Value Blank = Empty String ('') 0 is a value, not NULL
---	--	---

★ **Key Takeaway** : Understand the basics clearly - Everything in SQL builds on these concepts! 😊

2. DDL & DML COMMANDS



These commands are used to create, modify, manage and query table data.

		SYNTAX	EXAMPLE
1  CREATE (DDL) 	Used to create new table in database.	CREATE TABLE table_name (column1 datatype, column2 datatype, ...);	CREATE TABLE Student (id INT, name VARCHAR(50), age INT, course VARCHAR(50));
2  ALTER (DDL)	Used to modify existing table structure.	ALTER TABLE table_name ADD COLUMN column_name datatype; ALTER TABLE table_name MODIFY COLUMN column_name datatype; ALTER TABLE table_name DROP COLUMN column_name;	ALTER TABLE Student ADD email VARCHAR(50); ALTER TABLE Student MODIFY age INT; ALTER TABLE Student DROP COLUMN email;
 DROP (DDL)	Used to delete table completely from database.	DROP TABLE table_name;	DROP TABLE Student;
 TRUNCATE (DDL)	Used to remove all rows from table but keep structure.	TRUNCATE TABLE table_name;	TRUNCATE TABLE Student;
 INSERT (DML)	Used to insert new record(s) into table.	INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);	INSERT INTO Student (id, name, age, course) VALUES (1, 'Abhi', 22, 'CSE');
 UPDATE (DML)	Used to update existing record(s) in table.	UPDATE table_name SET column_name = value WHERE condition;	UPDATE Student SET age = 23 WHERE id = 1;
7 DELETE (DML)	Used to delete record(s) from table.	DELETE FROM table_name WHERE condition;	DELETE FROM Student WHERE id = 1;
8 SELECT (DQL)	Used to retrieve data from table.	SELECT column1, column2, ... FROM table_name WHERE condition;	SELECT * FROM Student WHERE age > 20;
 SELECT (DQL)	Used to retrieve with conditions, sorting, etc.	SELECT column1, column2, ... FROM table_name WHERE condition ORDER BY column1 ASC DESC;	SELECT name, age FROM Student WHERE course = 'CSE' ORDER BY age DESC;

KEY POINTS

- **DDL** = Structure-related commands (CREATE, ALTER, DROP, TRUNCATE)
- **DML** = Data-related commands (INSERT, UPDATE, DELETE)
- Always end statement with semicolon (;)
- SQL keywords are usually written in **UPPERCASE**.



Once table is **DROPPED**, data cannot be recovered.

- **TRUNCATE** is faster than **DELETE** because it doesn't log row-by-row in the same way.
- **DELETE** without **WHERE** can remove **ALL** rows!

DELETE vs TRUNCATE vs DROP

Feature	DELETE	TRUNCATE	DROP
Removes Data	Yes	Yes	Yes (Table)
Table Structure	Remains	Remains	Removed
Rollback	Yes	No	No
Speed	Slow	Fast	Fast
WHERE Clause	Yes	No	No

CRUD MEMORY TIP

- C** CREATE → Create data / table 
- R** READ → Retrieve data (SELECT) 
- U** UPDATE → Modify data 
- D** DELETE → Remove data 

INTERVIEW QUESTIONS

- 1 What is the difference between DELETE, TRUNCATE and DROP?
- 2 What is the difference between DDL and DML?
- 3 What happens if you run DELETE without WHERE clause?
- 4 Can we rollback after TRUNCATE command? Why or why not?



Practice these questions to crack any SQL interview with confidence!



3. DCL, TCL & CONSTRAINTS

DCL controls permissions and access. TCL manages transactions to ensure data consistency. Constraints enforce rules on data to maintain integrity and accuracy.

1) DCL - Data Control Language



DCL is used to control access and permissions on database objects (tables, views, procedures, etc.).

GRANT

Meaning : Gives privileges to users.

Syntax :

```
GRANT privilege_list ON object TO user [,user];
```

Example :

```
GRANT SELECT, INSERT ON Student TO john;
```

REVOKE

Meaning : Removes previously granted privileges.

Syntax :

```
REVOKE privilege_list ON object FROM user [,user];
```

Example :

```
REVOKE INSERT ON Student FROM john;
```

2) TCL - Transaction Control Language



TCL manages transactions to ensure data consistency.

★ COMMIT

- Meaning : Saves all changes permanently.
- Syntax : COMMIT;
- Example : COMMIT;

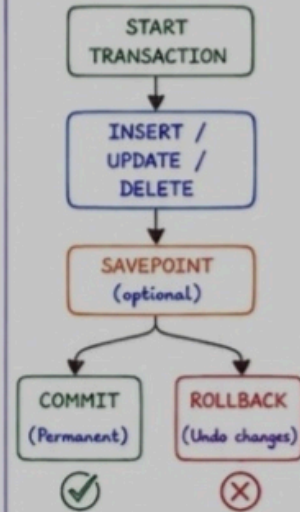
★ ROLLBACK

- Meaning : Undoes all changes since last COMMIT.
- Syntax : ROLLBACK;
- Example : ROLLBACK;

★ SAVEPOINT

- Meaning : Sets a point within transaction to rollback to.
- Syntax : SAVEPOINT savepoint_name;
ROLLBACK TO savepoint_name;
- Example : SAVEPOINT sp1;
ROLLBACK TO sp1;

Transaction Flow



3) CONSTRAINTS - Enforce Rules on Data



Constraint	Meaning	Syntax (Mini)	Example
NOT NULL	Column cannot have NULL value.	col_name DATA_TYPE NOT NULL	age INT NOT NULL
UNIQUE	All values in column must be unique.	col_name DATA_TYPE UNIQUE	email VARCHAR(100) UNIQUE
PRIMARY KEY	Uniquely identifies each row.	col_name DATA_TYPE PRIMARY KEY	id INT PRIMARY KEY
FOREIGN KEY	Maintains link between two tables.	col_name DATA_TYPE FOREIGN KEY REFERENCES parent_table(col_name)	dept_id INT FOREIGN KEY REFERENCES Department(dept_id)
CHECK	Ensures values satisfy a condition.	CONSTRAINT chk_name CHECK (condition)	age INT CHECK (age >= 18)
DEFAULT	Sets default value if not provided.	col_name DATA_TYPE DEFAULT default_value	status VARCHAR(10) DEFAULT 'Active'
AUTO_INCREMENT / IDENTITY	Auto generates unique numeric value.	col_name INT AUTO_INCREMENT	id INT AUTO_INCREMENT

4) KEYS - Quick Summary



KEY	DESCRIPTION
Primary Key	Uniquely identifies each row in a table. Cannot be NULL or duplicate.
Foreign Key	References Primary Key of another table to maintain relationship.
Candidate Key	A column (or set) that can uniquely identify rows. Primary key is one of them.



A table can have multiple candidate keys but only one primary key.

5) RELATIONSHIP - Child (Student) → Parent (Department)



Student (Child Table)				Department (Parent Table)	
(PK)	name	dept_id (FK)	age	dept_id (PK)	dept_name
1	Abhi	10	22	10	IT
2	Ravi	20	23	20	HR
3	Sita	10	21	30	Finance

References

Student.dept_id → Department.dept_id
(Foreign Key Relationship)

6) REMEMBER / BEST PRACTICES



- ✓ Constraints improve data quality.
- ✓ COMMIT persists changes permanently.
- ✓ ROLLBACK undoes uncommitted changes.
- ✓ SAVEPOINT allows partial rollback.
- ✓ Always test in a safe environment.



7) ACID REMINDER

Atomicity	All or nothing.
Consistency	DB remains valid before & after.
Isolation	Transactions do not interfere.
Durability	Committed data is permanent.

8) INTERVIEW QUESTIONS



2. TCL

- What is GRANT and REVOKE ?
- How do you give SELECT privilege on a table ?
- How can you revoke all privileges from a user ?
- What is the difference between COMMIT and ROLLBACK ?
- What is SAVEPOINT and how is it used ?
- Can we COMMIT a partial transaction ?

3. CONSTRAINTS

- What is the difference between PRIMARY KEY and UNIQUE ?
- When would you use CHECK constraint ?
- Can a table have multiple FOREIGN KEYS ?

4. GENERAL

- What is ACID property ?
- Why are constraints important in database design ?
- Give an example of parent - child relationship in SQL.

★ **Key Takeaway** : DCL secures data, TCL manages transactions, Constraints ensure integrity. 😊

4. SQL FUNCTIONS

★ Functions are predefined formulas in SQL that perform specific operations and return a value. ★

★ A. AGGREGATE FUNCTIONS (Work on multiple rows and return a single value)

Function	Description	Syntax	Example	Result
COUNT(*)	Counts number of rows	SELECT COUNT(*) FROM Employee;	SELECT COUNT(*) FROM Employee;	Total number of employees
SUM()	Returns sum of numeric values	SELECT SUM(Salary) FROM Employee;	SELECT SUM(Salary) FROM Employee;	Total Salary
AVG()	Returns average of numeric values	SELECT AVG(Salary) FROM Employee;	SELECT AVG(Salary) FROM Employee;	Average Salary
MAX()	Returns maximum value	SELECT MAX(Salary) FROM Employee;	SELECT MAX(Salary) FROM Employee;	Highest Salary
MIN()	Returns minimum value	SELECT MIN(Salary) FROM Employee;	SELECT MIN(Salary) FROM Employee;	Lowest Salary

KEY POINTS

- Aggregate functions operate over multiple rows.
- String functions manipulate character data.
- Numeric functions work with numbers.
- Date functions work with date and time.

★ B. STRING FUNCTIONS

Function	Description	Syntax	Example	Result
UPPER()	Converts string to upper case	SELECT UPPER(Name) FROM Employee;	SELECT UPPER('abhi');	ABHI
LOWER()	Converts string to lower case	SELECT LOWER(Name) FROM Employee;	SELECT LOWER('ABHI');	abhi
LENGTH()/LEN()	Returns length of string	SELECT LENGTH(Name) FROM Employee;	SELECT LENGTH('tajamul');	8
CONCAT()	Joins two or more strings	SELECT CONCAT(FirstName, ' ', LastName) FROM Employee;	SELECT CONCAT('Abhi', ' ', 'Khan');	Abhi Khan
SUBSTRING()	Extracts part of string	SELECT SUBSTRING(Name, start, length) FROM Employee;	SELECT SUBSTRING('tajamul', 1, 4);	taja

★ C. NUMERIC FUNCTIONS

Function	Description	Example	Result
ROUND()	Rounds to n decimal places	SELECT ROUND(123.456, 2);	123.46
CEIL()/CEILING()	Rounds up to nearest integer	SELECT CEIL(45.2);	46
FLOOR()	Rounds down to nearest integer	SELECT FLOOR(45.8);	45
ABS()	Returns absolute value	SELECT ABS(-25);	25

★ D. DATE FUNCTIONS

Function	Description	Example	Result (Sample)
NOW() / CURRENT_TIMESTAMP	Returns current date & time	SELECT NOW();	2025-05-28 14:30:25
CURDATE() / CURRENT_DATE	Returns current date	SELECT CURDATE();	2025-05-28
DATEDIFF()	Returns difference in days (date1 - date2)	SELECT DATEDIFF('2025-05-28', '2025-05-01');	27
DATEADD()	Adds interval to a date	SELECT DATEADD(DAY, 7, '2025-05-28');	2025-06-04

QUICK EXAMPLE QUERY

```
SELECT UPPER(Name) AS Name_Upper,
COUNT(*) AS Total_Employees,
AVG(Salary) AS Avg_Salary,
MAX(Salary) AS Max_Salary
FROM Employee
WHERE Department = 'IT';
```

Practice Makes Perfect!

NULL HANDLING IN AGGREGATES

- COUNT(*) counts all rows including NULLs.
- COUNT(column) counts only non-NULL values.
- SUM(), AVG(), MAX(), MIN() ignore NULLs.
- If all values are NULL, SUM/AVG return NULL.



TIP: Use COALESCE() or IFNULL() to handle NULL values.

FUNCTIONS ARE OFTEN NESTED ★

- Example : SELECT UPPER(SUBSTRING(Name, 1, 3)) FROM Employee;
- Example : SELECT ROUND(AVG(Salary), 2) FROM Employee;



Practice helps turn concepts into confidence! 😊

INTERVIEW QUESTIONS

1. What is the difference between COUNT(*) and COUNT(column)?
2. How does COUNT(column) behave with NULL values?
3. What is the difference between WHERE 'IT' and Department = 'IT'?

4. How does DATEDIFF() work? Give an example.

5. How can you get the last 3 characters of a string?

PREPARE WELL! ★

★ KEY TAKEAWAY : Functions make SQL powerful and expressive - Master them to write clean and efficient queries! 😊

5. FILTERING, SORTING & CLAUSES

These clauses help you retrieve the exact rows you need from your data. 💡

★ ① SELECT and DISTINCT

- ✓ SELECT returns columns.
- ✓ DISTINCT removes duplicate values.

Syntax :

```
SELECT column1, column2
FROM table_name;

SELECT DISTINCT column
FROM table_name;
```

Examples :

```
SELECT name, dept FROM Employee;

SELECT DISTINCT dept FROM
Employee;
```

★ ② WHERE Clause

Filters rows based on a condition.

Common Operators

Operator	Meaning	Example
=	Equal	age = 22
>	Greater than	salary > 50000
<	Less than	age < 30
>=	Greater or equal	salary >= 40000
<=	Less or equal	age <= 25
!= or <>	Not equal	dept != 'HR'

Syntax :

```
SELECT *
FROM table_name
WHERE condition;
```

★ ③ Logical Operators

Combine multiple conditions.

Syntax :

```
WHERE condition1 AND|OR|NOT condition2;
```

Examples :

- age > 20 AND dept = 'IT'
- salary > 50000 OR dept = 'HR'
- NOT (dept = 'HR')

AND
OR
NOT

⑦ Aliases (AS) for Columns and Tables

Aliases make result readable and queries easier to write.

Column Alias : →

```
SELECT name AS employee_name,
       salary AS monthly_salary
FROM Employee;
```

Table Alias : →

```
SELECT e.name, e.dept
FROM Employee e
WHERE e.salary > 50000;
```

★ ⑨ Simple Query Writing Flow



★ 4) Other Filtering Clauses

Clause	What it does	Example
BETWEEN	Selects values within a range (inclusive).	age BETWEEN 20 AND 30
IN	Matches any value in a list.	dept IN ('IT', 'HR', 'Finance')
LIKE	Pattern matching using % and _.	name LIKE 'A%' name LIKE '%an_'
IS NULL	Checks for NULL values.	manager IS NULL
IS NOT NULL	Checks for non-NULL values.	manager IS NOT NULL

★ ⑤ ORDER BY Clause

Sorts result set by one or more columns.

Syntax :

```
SELECT column1, column2
FROM table_name
ORDER BY column1 [ASC|DESC],
       column2 [ASC|DESC];
```

Example :

```
SELECT name, salary
FROM Employee
ORDER BY salary DESC,
       name ASC;
```

★ ⑥ LIMIT / TOP / FETCH FIRST

Limits the number of rows returned.

Syntax :

```
-- MySQL / SQLite :
SELECT * FROM Employee LIMIT 5;

-- SQL Server :
SELECT TOP 5 * FROM Employee;

-- Oracle / PostgreSQL :
SELECT * FROM Employee FETCH FIRST 5 ROWS ONLY;
```

Note :

Syntax varies across databases. Use the one supported by your DB. 😊

★ ⑧ Quick Comparison & Tips

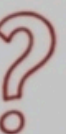
- ✓ WHERE filters rows **BEFORE** grouping/aggregation.
- ✓ LIKE uses % (any sequence) and _ (single char) wildcards.
- ✓ BETWEEN is inclusive of both ends.
- ✓ IN is easier than multiple OR conditions.
- ✓ NULL is checked with **IS NULL** / **IS NOT NULL**, not = NULL.

Tip:

Read the question carefully and choose the right clause! 😊

★ ⑩ Interview / Common Mistakes

1. What is the difference between WHERE and HAVING?
2. Explain the use of BETWEEN, IN and LIKE with examples.
3. How do you handle NULL values in SQL?
4. Why is ORDER BY clause applied after WHERE clause?



★ **Key Takeaway :** Use filtering, sorting and clauses to get the exact results you need — fast and accurate! 😊

6. JOINS IN SQL

★ Joins combine rows from two or more tables based on a related column between them. They help us get meaningful data from multiple tables.

INNER JOIN

Returns only the matching rows in both tables.

Syntax :

```
SELECT * FROM A
INNER JOIN B
ON A.key = B.key;
```

Example :

```
SELECT * FROM Customers C
INNER JOIN Orders O
ON C.id = O.customer_id;
```

LEFT JOIN (LEFT OUTER JOIN)

Returns all rows from the left table and the matching rows from the right table. If no match, right side is NULL.

Syntax :

```
SELECT * FROM A
LEFT JOIN B
ON A.key = B.key;
```

Example :

```
SELECT * FROM Customers C
LEFT JOIN Orders O
ON C.id = O.customer_id;
```

RIGHT JOIN (RIGHT OUTER JOIN)

Returns all rows from the right table and the matching rows from the left table. If no match, left side is NULL.

Syntax :

```
SELECT * FROM A
RIGHT JOIN B
ON A.key = B.key;
```

Example :

```
SELECT * FROM Customers C
RIGHT JOIN Orders O
ON C.id = O.customer_id;
```

FULL OUTER JOIN

Returns all rows when there is a match in either left or right table. If no match on either side, fills NULLs.

Syntax :

```
SELECT * FROM A
FULL OUTER JOIN B
ON A.key = B.key;
```

Example :

```
SELECT * FROM Customers C
FULL OUTER JOIN Orders O
ON C.id = O.customer_id;
```

SELF JOIN

A table is joined with itself. Useful for hierarchical or recursive relationships.

Syntax :

```
SELECT * FROM Table T1
JOIN Table T2
ON T1.col = T2.col;
```

Example Use Case :

```
-- Employees and their Managers
SELECT E.name, M.name AS Manager
FROM Employees E
LEFT JOIN Employees M ON E.mgr_id = M.id;
```

CROSS JOIN

Returns the Cartesian product of both tables (every row from A with every row from B).

Syntax :

```
SELECT * FROM A
CROSS JOIN B;
```

Example :

```
SELECT * FROM Customers
CROSS JOIN Regions;
```

JOIN COMPARISON

Join Type	What it Returns
INNER JOIN	Rows with matches in both tables only
LEFT JOIN	All rows from left table + matching from right
RIGHT JOIN	All rows from right table + matching from left
FULL OUTER JOIN	All rows from both tables (matched or not)
SELF JOIN	Table joined with itself
CROSS JOIN	Cartesian product (every row of A with B)

REAL-WORLD USE CASES

- Customers + Orders → Get order details for customers. 
- Students + Departments → List students with their departments. 
- Employees + Salaries → See salary details with employee info. 
- Products + Categories → Show products with category names. 

7) JOIN DIAGRAM & EXAMPLE

Customers (C)

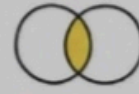
id	name	city
1	Abhi	Delhi
2	Ravi	Mumbai
3	Sita	Pune

Orders (O)

id	customer_id	amount
101	1	2500
102	1	1500
103	3	3000
104	4	4000

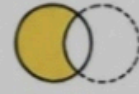
Join Intuition (based on C.id = O.customer_id)

INNER JOIN



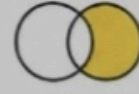
Only matching rows

LEFT JOIN



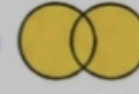
All from left + match

RIGHT JOIN



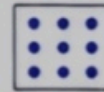
All from right + match

FULL OUTER JOIN



All from both sides

CROSS JOIN



All combinations

INNER JOIN OUTPUT (Example)

C.id	C.name	O.id	O.amount
1	Abhi	101	2500
1	Abhi	102	1500
3	Sita	103	3000

★ 9) COMMON MISTAKES / TIPS

- Always specify JOIN condition using **ON**.
- Missing **ON** leads to **CROSS JOIN** (huge result!).
- Be careful with duplicate rows (one-to-many).
- Use table aliases to keep queries readable.
- Difference between **WHERE** and **ON** in joins:
 - **ON** filters rows before join.
 - **WHERE** filters rows after join.

11) INTERVIEW QUESTIONS

- ① What is the difference between INNER JOIN and LEFT JOIN ?
- ② What is the result of a CROSS JOIN ?
- ③ When would you use SELF JOIN ? Give an example.
- ④ Explain the difference between WHERE and ON in JOINS.

Remember : Joins are powerful — use them wisely and always check your results! ❤️

7. GROUP BY, HAVING & CASE

These features help us summarize data and build powerful logic in queries.

1 GROUP BY

- Used to group rows that have the same values in one or more columns.
- Usually used with aggregate functions.

Syntax :

```
SELECT column1, AGG_FUNC(column2)
FROM table_name
GROUP BY column1, column2, ... ;
```

Example Query :

```
SELECT Dept, COUNT(*) AS Total_Employees,
SUM(Salary) AS Total_Salary,
AVG(Salary) AS Avg_Salary
FROM Employee
GROUP BY Dept ;
```

Example : Total Salary per Department

Dept	Total_Employees	Total_Salary	Avg_Salary
IT	3	147000	49000.00
HR	2	97000	48500.00
Finance	2	69000	34500.00

2 HAVING

- Used to filter groups (works after GROUP BY).
- Cannot use aggregate functions in WHERE.

Syntax :

```
SELECT column1, AGG_FUNC(column2)
FROM table_name
GROUP BY column1
HAVING condition ;
```

Example : Departments with more than 2 employees

```
SELECT Dept, COUNT(*) AS Total_Employees
FROM Employee
GROUP BY Dept
HAVING COUNT(*) > 2 ;
```

3 CASE WHEN

- Used to create custom labels or perform conditional logic.
- Returns a value based on conditions.

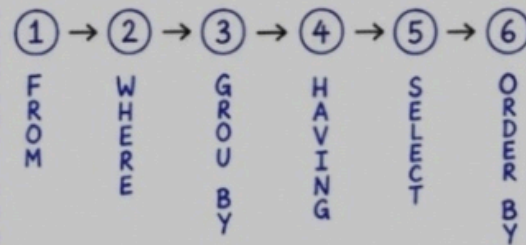
Syntax :

```
CASE
WHEN condition1 THEN result1
WHEN condition2 THEN result2
...
ELSE resultN
END AS alias
```

Example : Salary Level

```
SELECT Name, Salary,
CASE
WHEN Salary >= 70000 THEN 'High'
WHEN Salary >= 40000 THEN 'Medium'
ELSE 'Low'
END AS Salary_Level
FROM Employee ;
```

4 ORDER OF EXECUTION



5 AGGREGATE COMBINATIONS

Function	Meaning	Works With GROUP BY?
COUNT(*)	Count rows	✓
COUNT(column)	Count non-NULL values	✓
SUM(column)	Sum of values	✓
AVG(column)	Average of values	✓
MAX(column)	Maximum value	✓
MIN(column)	Minimum value	✓

6 WHERE vs HAVING

Aspect	WHERE (Row Filter)	HAVING (Group Filter)
When Applied	Filters rows before grouping	Filters groups after grouping
Aggregate Functions	Cannot use aggregate functions	Can use aggregate functions
Works On	Individual rows	Groups
Performance	Improves performance	Slightly slower (after grouping)
Usage	Used with or without GROUP BY	Used only with GROUP BY

BEST PRACTICES

- ★ Every non-aggregated column in SELECT must appear in GROUP BY.
- ★ WHERE filters rows; HAVING filters groups.
- ★ Use meaningful aliases for readability.
- ★ CASE is powerful inside SELECT, ORDER BY and aggregate expressions.
- ★ Keep queries simple, readable and optimized!

7 RAW DATA vs GROUPED SUMMARY

Raw Rows (Employee)

ID	Name	Dept	Salary
1	Abhi	IT	50000
2	Ravi	IT	45000
3	Sita	HR	52000
4	Neha	HR	45000
5	Omar	Finance	30000
6	Priya	Finance	39000

GROUP BY
Dept

Grouped Summary (by Dept)

Dept	Emp_Count	Total_Salary	Avg_Salary
Finance	2	69000	34500.00
HR	2	97000	48500.00
IT	2	95000	47500.00

8 INTERVIEW QUESTIONS

- What is the difference between WHERE and HAVING ?
• Give an example. 
- Write a query to find departments with more than 2 employees. 
- Write a query to show employee name, salary and salary level (High/Medium/Low) using CASE WHEN. 
- Write a query to find the department with highest average salary. 

8. SUBQUERIES, CTEs & SET OPERATORS

Powerful tools to break complex SQL into manageable parts, improve readability and combine result sets. ★

★ 1) SUBQUERIES

A subquery is a query inside another query. 🔍

Type	Description	Example
Single-Row Subquery	Returns exactly one row (one value).	<code>SELECT name FROM Employee WHERE id = (SELECT MAX(id) FROM Employee);</code>
Multi-Row Subquery	Returns multiple rows (multiple values).	<code>SELECT name FROM Employee WHERE dept_id IN (SELECT id FROM Department);</code>
Correlated Subquery	Refers to columns from outer query. Executes for each row.	<code>SELECT name FROM Employee e1 WHERE salary > (SELECT AVG(salary) FROM Employee e2 WHERE e2.dept_id = e1.dept_id);</code>

★ 2) EXISTS vs IN

EXISTS	IN
- Returns TRUE if subquery returns any row. - Often better with large result sets. Example: <pre>SELECT name FROM Employee e WHERE EXISTS (SELECT 1 FROM Department d WHERE d.id = e.dept_id);</pre>	- Compares value with a list returned by subquery. - Good for smaller lists. Example: <pre>SELECT name FROM Employee WHERE dept_id IN (SELECT id FROM Department);</pre>

💡 Rule of Thumb: Use **EXISTS** for existence check, **IN** for value comparison.

★ 3) COMMON TABLE EXPRESSIONS (CTEs)

CTE is a temporary named result set defined using WITH clause.

Syntax:	Use Case:
<pre>WITH cte_name AS (SELECT column1, column2, ... FROM table_name WHERE condition) SELECT * FROM cte_name WHERE ...;</pre>	✓ Improve readability. ✓ Reuse a result set. ✓ Break complex queries into logical steps. Example: <pre>WITH HighSalary AS (SELECT * FROM Employee WHERE salary > 50000) SELECT name, dept_id FROM HighSalary WHERE dept_id = 1;</pre>

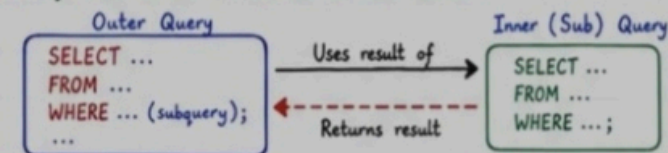
★ 4) SET OPERATORS

Operator	Description	Example
UNION	Combines results and removes duplicates.	<code>SELECT id FROM A UNION SELECT id FROM B;</code>
UNION ALL	Combines results and keeps duplicates.	<code>SELECT id FROM A UNION ALL SELECT id FROM B;</code>
INTERSECT	Returns only common rows in both queries.	<code>SELECT id FROM A INTERSECT SELECT id FROM B;</code>
EXCEPT / MINUS	Returns rows in first query not in second.	<code>SELECT id FROM A EXCEPT SELECT id FROM B;</code> (In Oracle use MINUS)

UNION vs UNION ALL

UNION	UNION ALL
✓ Removes duplicates ✓ Slower (deduplication cost) ✓ Use when unique rows needed	✓ Keeps duplicates ✓ Faster ✓ Use when all rows needed

★ 5) SUBQUERY NESTING DIAGRAM



★ 6) WHEN TO USE WHAT?

- ✓ Subqueries - For nested filtering, comparisons, existence checks.
- ✓ CTEs - For better readability, reusable logic, and breaking query into steps.
- ✓ UNION / UNION ALL - To combine similar result sets from multiple queries.
- ✓ EXISTS - When you only need to check if related data exists. 😊

★ 7) COMMON MISTAKES

- ✗ Mismatched column count in UNION/INTERSECT.
- ✗ Using UNION when duplicates are required.
- ✗ Forgetting correlated reference in correlated subquery (e2.col = e1.col).
- ✗ Using subquery in SELECT list that returns multiple rows.
- ✗ ORDER BY inside subquery without LIMIT / FETCH (may not work in all DBs).
- ✗ Placing ORDER BY before UNION (should be at the end).

★ 8) QUICK EXAMPLES (COMBO)

CTE + EXISTS:

```
WITH DeptEmp AS (
  SELECT dept_id, COUNT(*) cnt
  FROM Employee
  GROUP BY dept_id
)
SELECT d.dept_id FROM DeptEmp d
WHERE EXISTS (SELECT 1 FROM Department dp
              WHERE dp.id = d.dept_id);
```

UNION ALL Example:

```
SELECT name, 'Employee' AS type FROM Employee
UNION ALL
SELECT name, 'Manager' AS type FROM Manager;
```

9) INTERVIEW QUESTIONS

- What is a subquery? What are the types of subqueries with examples? 🤔
- What is the difference between EXISTS and IN? When would you use each? ⚖️
- What is a CTE? Write a query using CTE to find the top 3 highest paid employees in each department. 📊
- Explain UNION, UNION ALL, INTERSECT and EXCEPT with examples. What is the difference between UNION and UNION ALL? 💡

♥ Practice daily, write queries often, and think in sets! ★

Happy Querying! 😊